Use Case Descriptions for FamPlan

Lab 3 - Complete Use Case Model

Version 1.0 (aligned to SRS v1.5)

Prepared by Banana Five-O

NTU SC2006 Software Engineering

16 October 2026

1. Introduction

This document completes the FamPlan use case model for Lab 3. Section 2 gives the use case diagram and Section 3 describes all 16 use cases in the SRS v1.5 catalogue (Section 7.5), in the same template as the Lab 1 descriptions. Each description traces to the functional requirements in SRS Section 4 and the non-functional requirements in SRS Section 5, and names the prototype screen or endpoint that implements it.

Conventions. Main flows have at most seven steps. Alternative flows are numbered AF-<step><letter>; for example, AF-4a branches from main-flow step 4. "System" means the FamPlan application. Inherited associations are not repeated: a Registered User can also perform every Visitor use case, and a Data Administrator every Registered User use case.

1.1 Actors

Actor	Type	Description
Visitor	Primary	Any member of the public using FamPlan without signing in.
Registered User	Primary	A signed-in household member; inherits every Visitor use case.
Data Administrator	Primary	A Registered User with the DATA_ADMINISTRATOR role; monitors automatic synchronisation only.
Email Delivery Service	Supporting	Sends password-reset emails (UC-1.3).
OneMap	Supporting	Singapore Land Authority map, geocoding and routing service (UC-5.1, UC-5.2).
Configured Public Data Sources	Supporting	data.gov.sg (URA, ECDA, MOE, MOH, NParks, LTA) and OSM datasets synchronised automatically (UC-4.2, UC-5.1, UC-7.1).

1.2 Use case catalogue

ID	Use case	Actor(s)	Requirements
UC-1.1	Register Account	Visitor	FR-ACC-01
UC-1.2	Login and Logout	Registered User	FR-ACC-02, FR-ACC-03
UC-1.3	Reset Password	Registered User (primary); Email Delivery Service (supporting)	FR-ACC-04
UC-1.4	Manage Account Details	Registered User	FR-ACC-05
UC-1.5	Delete Account	Registered User	FR-ACC-06
UC-2.1	Manage Family Preferences	Registered User	FR-PREF-01 to FR-PREF-07
UC-3.1	Search Neighbourhoods	Visitor	FR-DISC-01, FR-DISC-02, FR-DISC-04 to FR-DISC-06
UC-3.2	Apply Saved Preferences	Registered User	FR-DISC-03
UC-4.1	View Neighbourhood Profile	Visitor	FR-NBH-01; FR-DEC-01 to FR-DEC-03
UC-4.2	Explore Family Amenities	Visitor (primary); Configured Public Data Sources (supporting)	FR-NBH-02 to FR-NBH-10

UC-5.1	View Map and Nearby Places	Visitor (primary); Configured Public Data Sources and OneMap (supporting)	FR-NBH-02 to FR-NBH-09; FR-TRV-01
UC-5.2	Check Commute Route	Visitor (primary); OneMap (supporting)	FR-TRV-02 to FR-TRV-05
UC-6.1	Calculate Family Suitability Score	None (included behaviour)	FR-DEC-01 to FR-DEC-03
UC-6.2	Compare Neighbourhoods	Visitor	FR-DEC-04; FR-DEC-01 to FR-DEC-03
UC-6.3	Manage Shortlist and Notes	Registered User	FR-DEC-05 to FR-DEC-07
UC-7.1	Monitor Automatic Data Synchronisation	Data Administrator (primary); Configured Public Data Sources (supporting)	FR-ADM-01, FR-ADM-02, FR-ADM-04 (background: FR-ADM-03, FR-RES-01)

2. Use Case Diagram

Figure 1 shows the 16 use cases in their seven SRS groups. UC-6.1 has no initiating actor because it is included by UC-4.1 and UC-6.2.

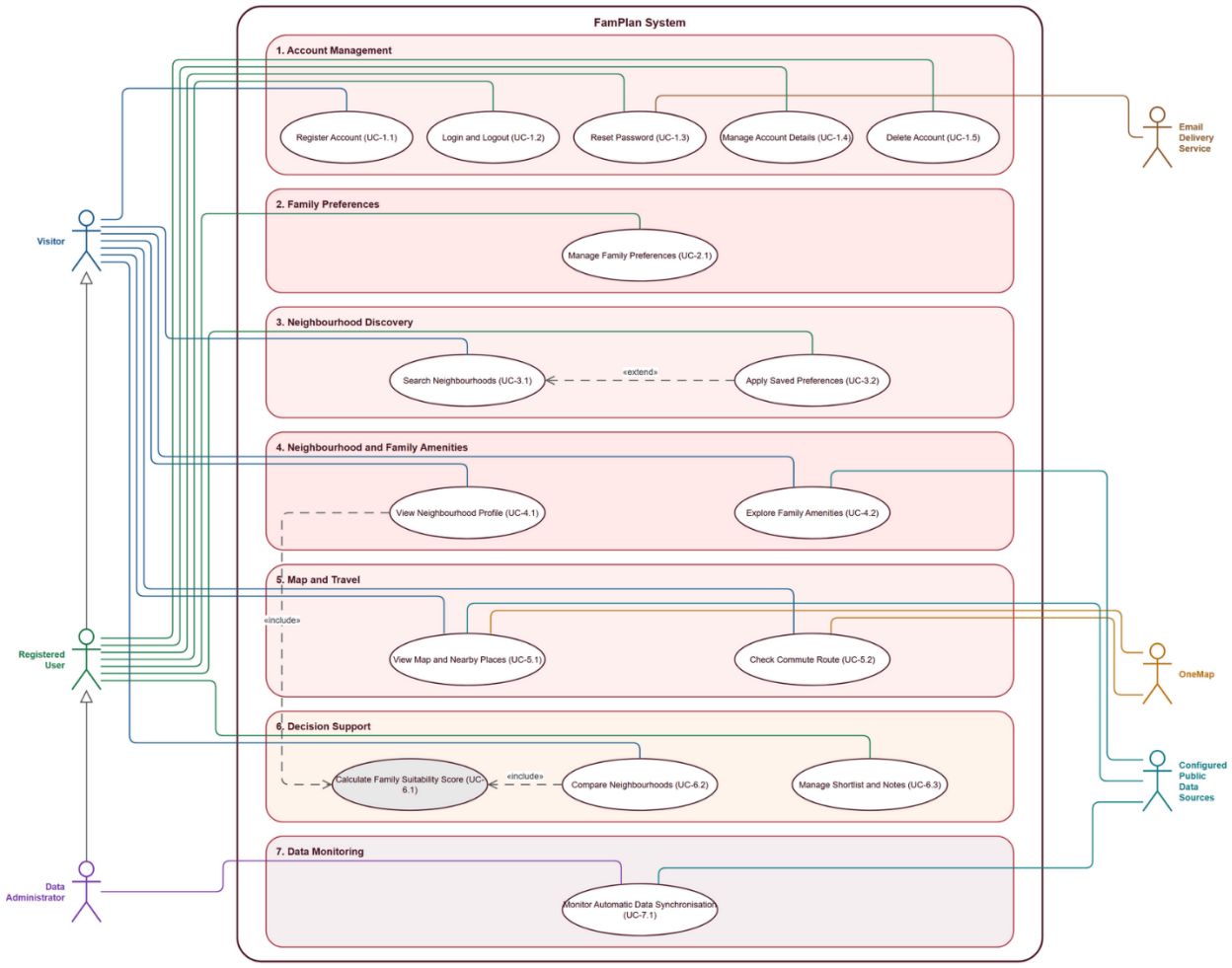


Figure 1. FamPlan use case diagram (16/10/2026)

3. Use Case Descriptions

Each use case starts on a new page, grouped as in SRS Section 7.2: 1 Account Management, 2 Family Preferences, 3 Neighbourhood Discovery, 4 Neighbourhood and Family Amenities, 5 Map and Travel, 6 Decision Support, 7 Data Monitoring.

UC-1.1 Register Account

Use Case ID:	UC-1.1
Use Case Name:	Register Account
Actor(s):	Visitor
Description:	Lets a visitor create a FamPlan account using a display name, email address and password, after explicitly consenting to the stated data use.
Preconditions:	1. The visitor is not signed in.
Postconditions:	• Success: the account, an empty family preference profile and a private household workspace are created, and the visitor is signed in as a Registered User. • Failure: no account is created; field-specific errors are shown.
Priority:	Must
Frequency of Use:	Low (once per user)
Flow of Events:	1. Visitor opens the Sign in page and selects the Create account tab. 2. Visitor enters a display name, email address and password. 3. Visitor ticks the data-use consent statement and selects Create account. 4. System validates every field, the password policy and the consent. 5. System creates the account and stores only a salted Argon2id hash of the password. 6. System starts an authenticated session and shows the signed-in view.
Alternative Flows:	• AF-4a Email already registered: System shows "An account with this email already exists. Sign in or reset your password."; return to step 2. • AF-4b Malformed email or password that breaks the policy: System shows a field error naming the rule; return to step 2. • AF-4c Consent not given: System shows a consent error; return to step 3.
Exceptions:	• Too many attempts or service unavailable: System shows a generic error and creates no account.
Includes / Extends:	None
Requirements Traced:	FR-ACC-01
Special Requirements:	NFR-SEC-01 (HTTPS only); NFR-SEC-02 (salted password hash); NFR-SEC-05 (input validation); NFR-PRIV-02 (no exact dates of birth or unnecessary personal data).
Assumptions:	The email address uniquely identifies an account; there is no separate email-verification step in this release.
Notes and Issues:	Implemented in client/src/pages/SignIn.tsx and POST /auth/register (server/src/account/routes.ts).

UC-1.2 Login and Logout

Use Case ID:	UC-1.2
Use Case Name:	Login and Logout
Actor(s):	Registered User
Description:	Lets a registered user start an authenticated session with email and password, and end that session.
Preconditions:	1. Login: the user has a registered account and is not signed in. 2. Logout: the user is signed in.
Postconditions:	• Login success: a session is created (12-hour lifetime) and stored in an httpOnly cookie. • Login failure: a generic failure is shown and no session is created. • Logout: the session is invalidated on the server and the user is returned to a public page.
Priority:	Must
Frequency of Use:	High (every visit)
Flow of Events:	1. User opens the Sign in page. 2. User enters email and password and selects Sign in. 3. System verifies the password against the stored hash and checks that the account is not locked. 4. System creates a session, resets the failed-login counter and shows the signed-in view. 5. Later, the user selects Log out. 6. System invalidates the session on the server and returns the user to a public page.
Alternative Flows:	• AF-3a Wrong password or unknown email: System shows the same generic failure for both, so account existence is not revealed, and increments the failed-login counter; return to step 2. • AF-3b Fifth consecutive failure: System locks password sign-in for 15 minutes and shows the generic failure. • AF-5a Session expired: protected pages ask the user to sign in again.
Exceptions:	• Service unavailable: System shows a generic error and creates no session.
Includes / Extends:	None
Requirements Traced:	FR-ACC-02, FR-ACC-03
Special Requirements:	NFR-SEC-01 (HTTPS only); NFR-SEC-02 (password hashing); NFR-SEC-04 (15-minute lockout after five failures).
Assumptions:	Users sign in from a browser that accepts first-party cookies.
Notes and Issues:	Implemented in client/src/pages/SignIn.tsx, POST /auth/login and /auth/logout, and server/src/account/sessions.ts.

UC-1.3 Reset Password

Use Case ID:	UC-1.3
Use Case Name:	Reset Password
Actor(s):	Registered User (primary); Email Delivery Service (supporting)
Description:	Lets a user who has forgotten the password receive a time-limited, single-use reset link by email and choose a new password.
Preconditions:	1. The user is not signed in.
Postconditions:	• Success: the new password hash is stored, the link is marked used, all existing sessions are ended and any lockout is cleared. • Failure: the password is unchanged.
Priority:	Should
Frequency of Use:	Low
Flow of Events:	1. User selects Forgot password and enters the account email address. 2. System shows the same generic confirmation whether or not the email is registered. 3. If the email is registered, System asks the Email Delivery Service to send a reset link valid for 30 minutes. 4. User opens the link, enters a new password and confirms it. 5. System checks that the link is valid, unused and unexpired, and validates the password policy. 6. System stores the new password hash, marks the link used, ends existing sessions and asks the user to sign in.
Alternative Flows:	• AF-5a Link invalid, already used or expired: System shows "This reset link is invalid, already used, or expired. Request a new link."; return to step 1. • AF-5b Password breaks the policy or the confirmation does not match: System shows a field error; return to step 4.
Exceptions:	• Email delivery fails: no link arrives; the user can request a new link from step 1.
Includes / Extends:	None
Requirements Traced:	FR-ACC-04
Special Requirements:	NFR-SEC-02 (password hashing); NFR-SEC-04 (a successful reset clears the lockout); NFR-SEC-05 (input validation).
Assumptions:	The user still has access to the registered email inbox.
Notes and Issues:	Implemented in client/src/pages/PasswordReset.tsx, POST /auth/forgot-password and /auth/reset-password, and server/src/account/mailer.ts.

UC-1.4 Manage Account Details

Use Case ID:	UC-1.4
Use Case Name:	Manage Account Details
Actor(s):	Registered User
Description:	Lets a signed-in user change the display name stored on the account. Email address and password cannot be changed through this function.
Preconditions:	1. The user is signed in.
Postconditions:	• Success: the new display name is stored and shown. • Failure: the display name is unchanged and a field error is shown.
Priority:	Must
Frequency of Use:	Low
Flow of Events:	1. User opens Account settings. 2. System shows the current display name and the read-only email address. 3. User edits the display name and saves. 4. System validates the new name. 5. System stores it and confirms the change.
Alternative Flows:	• AF-4a Name empty or longer than 60 characters: System shows a field error; return to step 3.
Exceptions:	• Session expired: System asks the user to sign in again.
Includes / Extends:	None
Requirements Traced:	FR-ACC-05
Special Requirements:	NFR-SEC-05 (input validation).
Assumptions:	None.
Notes and Issues:	Implemented in client/src/pages/Account.tsx and PATCH /account.

UC-1.5 Delete Account

Use Case ID:	UC-1.5
Use Case Name:	Delete Account
Actor(s):	Registered User
Description:	Permanently deletes the account and all associated personal data after explicit confirmation.
Preconditions:	1. The user is signed in.
Postconditions:	• Success: the account, credentials, sessions, saved preferences, priority destinations, and owned shortlist and notes are deleted; the session ends and an acknowledgement is shown. • Cancelled: nothing changes.
Priority:	Must
Frequency of Use:	Very low
Flow of Events:	1. User opens Account settings and selects Delete account. 2. System explains exactly what will be deleted and that deletion cannot be undone. 3. User types DELETE to confirm and submits. 4. System deletes the account and all associated personal data. 5. System ends the session and shows a deletion acknowledgement on a public page.
Alternative Flows:	• AF-3a User cancels: nothing is deleted. • AF-4a Confirmation text is not DELETE: System shows "Type DELETE to confirm account deletion."; return to step 3.
Exceptions:	• Service unavailable: no data is deleted; System shows a generic error.
Includes / Extends:	None
Requirements Traced:	FR-ACC-06
Special Requirements:	NFR-PRIV-01 (deleted personal data is no longer retrievable).
Assumptions:	None.
Notes and Issues:	Implemented in client/src/pages/Account.tsx and DELETE /account.

UC-2.1 Manage Family Preferences

Use Case ID:	UC-2.1
Use Case Name:	Manage Family Preferences
Actor(s):	Registered User
Description:	Lets a user save family-planning stages, preferred planning areas, up to three priority destinations, amenity distance thresholds and eight criterion weights, with optional stage-based suggested weights.
Preconditions:	1. The user is signed in (needed to save).
Postconditions:	• Success: the preference profile is saved and used by search, scoring and comparison. • Failure: the previously saved profile is kept and field errors are shown.
Priority:	Must
Frequency of Use:	Medium (at setup, then occasionally)
Flow of Events:	1. User opens Family preferences. 2. System loads the saved profile, or defaults for a new account. 3. User selects family stages and preferred planning areas. 4. User adds up to three priority destinations (label and address) and sets amenity distances and the eight criterion weights (0 to 5). 5. Optionally, user requests suggested weights; System fills stage-based presets without overwriting weights the user has already confirmed. 6. User saves; System validates every value. 7. System stores the profile and confirms the save.
Alternative Flows:	• AF-1a Visitor not signed in: weights can be tried for the current session only, and System prompts the user to sign in to save. • AF-4a A fourth destination is added: System rejects it (maximum three). • AF-6a Weight outside 0 to 5 or not a whole number, or an unresolved destination address: System shows a field error; return to step 4.
Exceptions:	• Address search unavailable: destinations cannot be added, but the other preferences can still be saved.
Includes / Extends:	Saved values are reused by UC-3.2 Apply Saved Preferences.
Requirements Traced:	FR-PREF-01 to FR-PREF-07
Special Requirements:	NFR-PRIV-02 (family stages only; no exact dates of birth); NFR-SEC-05 (input validation).
Assumptions:	A household is represented by a single preference profile.
Notes and Issues:	Implemented in client/src/pages/Preferences.tsx, GET/PUT /preferences and POST /preferences/suggest-weights. Issue for review: destination address suggestions come from OneMap (GET /geocode), but SRS §7.3 does not associate OneMap with UC-2.1.

UC-3.1 Search Neighbourhoods

Use Case ID:	UC-3.1
Use Case Name:	Search Neighbourhoods
Actor(s):	Visitor
Description:	Finds neighbourhoods by town, planning area, estate, address or postal code, and filters and sorts the results.
Preconditions:	1. Validated neighbourhood data has been loaded.
Postconditions:	• Success: matching neighbourhood profiles are listed and plotted on the map, each labelled as a neighbourhood profile rather than a property listing. • Failure: an empty state or field-specific correction is shown.
Priority:	Must
Frequency of Use:	High
Flow of Events:	1. Visitor opens Discover neighbourhoods. 2. Visitor enters a town, estate, address or postal code, or picks planning areas. 3. Visitor optionally sets filters (family stage, amenity category, minimum facility count, facility radius, commute destination and time) and a sort order. 4. System validates the criteria. 5. System searches the locally stored validated data and orders the matches by the chosen sort. 6. System lists the matching neighbourhood profiles and plots them on the map.
Alternative Flows:	• AF-2a Spelling variant: System still matches the intended area by approximate name matching. • AF-3a Registered User selects Apply my preferences: UC-3.2 extends the search at this point. • AF-4a Invalid or contradictory criteria: System shows field-specific correction guidance; return to step 3. • AF-6a No matches: System shows an empty state suggesting wider filters.
Exceptions:	• Source data older than the freshness threshold: results are still shown with a stale-data warning.
Includes / Extends:	Extended by UC-3.2 (extension point: step 3, Apply my preferences).
Requirements Traced:	FR-DISC-01, FR-DISC-02, FR-DISC-04 to FR-DISC-06
Special Requirements:	NFR-PERF-01 (search response time); NFR-REL-03 (search reads local validated data, never live APIs); NFR-SEC-05 (input validation).
Assumptions:	Neighbourhoods are planning-area subzones from the URA dataset.
Notes and Issues:	Implemented in client/src/pages/Explore.tsx and GET /neighbourhoods (searchNeighbourhoods() in server/src/neighbourhoods/service.ts).

UC-3.2 Apply Saved Preferences

Use Case ID:	UC-3.2
Use Case Name:	Apply Saved Preferences
Actor(s):	Registered User
Description:	Fills the current search criteria from the user's saved family preferences and refreshes the results.
Preconditions:	1. The user is signed in. 2. A search is in progress (UC-3.1).
Postconditions:	• Search criteria are populated and results refreshed; the saved preference values are not changed.
Priority:	Must
Frequency of Use:	Medium
Flow of Events:	1. During a search, the user selects Apply my preferences. 2. System loads the saved family stages, preferred areas, amenity distances, destination and weights. 3. System fills the search criteria with these values. 4. System re-runs the search (UC-3.1 steps 4 to 6). 5. System confirms that the preferences were applied without changing the saved values.
Alternative Flows:	• AF-1a User not signed in: the control reads "Sign in to apply saved family preferences". • AF-2a Profile only partly filled: System applies the saved fields and keeps the current values for the rest. • AF-2b No saved profile: System directs the user to Family preferences (UC-2.1).
Exceptions:	• None.
Includes / Extends:	Extends UC-3.1.
Requirements Traced:	FR-DISC-03
Special Requirements:	None.
Assumptions:	None.
Notes and Issues:	Implemented in client/src/pages/Explore.tsx (applyMyPreferences()).

UC-4.1 View Neighbourhood Profile

Use Case ID:	UC-4.1
Use Case Name:	View Neighbourhood Profile
Actor(s):	Visitor
Description:	Shows a selected neighbourhood's summary (name, planning area, map location and coverage basis) together with its explainable family suitability score.
Preconditions:	1. A neighbourhood has been selected from results, a comparison, the shortlist or a direct link.
Postconditions:	• The profile and score breakdown are displayed.
Priority:	Must
Frequency of Use:	High
Flow of Events:	1. Visitor selects a neighbourhood. 2. System loads it from the local validated data. 3. System displays its name, planning area, map location and boundary or coverage basis. 4. System calculates its family suitability score (includes UC-6.1). 5. System shows the score with its criterion breakdown and the statement that the score is a comparison aid, not a guarantee. 6. System shows the data sources and their update times.
Alternative Flows:	• AF-2a Unknown neighbourhood: System shows "not found" with a link back to search. • AF-4a A criterion lacks data: the score discloses how its weight was treated (see UC-6.1).
Exceptions:	• Cached data older than the freshness threshold: System shows its age and a stale-data warning.
Includes / Extends:	Includes UC-6.1.
Requirements Traced:	FR-NBH-01; FR-DEC-01 to FR-DEC-03
Special Requirements:	NFR-TRANS-01 and NFR-TRANS-02 (explainable score, limitation statement); NFR-DATA-01 (provenance on every result).
Assumptions:	None.
Notes and Issues:	Implemented in client/src/pages/Profile.tsx and GET /neighbourhoods/:id.

UC-4.2 Explore Family Amenities

Use Case ID:	UC-4.2
Use Case Name:	Explore Family Amenities
Actor(s):	Visitor (primary); Configured Public Data Sources (supporting)
Description:	Lets the visitor review nearby childcare centres, kindergartens, schools, supermarkets, clinics, parks, MRT stations and bus stops within a chosen radius, with counts, nearest facilities, distances and sources.
Preconditions:	1. A neighbourhood profile is open (UC-4.1).
Postconditions:	• Per-category facilities, counts and availability states are displayed.
Priority:	Must
Frequency of Use:	High
Flow of Events:	1. Visitor chooses a radius (500 m, 1 km or 2 km) and optionally an amenity category. 2. System selects facilities within the radius from the validated datasets synchronised from the configured public data sources. 3. System shows the count for each category and the nearest facility with its distance and calculation basis. 4. Visitor opens a facility. 5. System shows its name, category, address where available, source and update time. 6. For schools and childcare, System states that proximity does not guarantee admission, enrolment, vacancy or eligibility.
Alternative Flows:	• AF-2a The same facility appears in two sources: System counts it once. • AF-3a No facility within the radius: System shows an explicit "no match" state instead of a misleading zero. • AF-3b A category's source is unavailable, incomplete or stale: System shows that status for the category; other categories remain visible.
Exceptions:	• None beyond AF-3b; data is served from the local validated store.
Includes / Extends:	None
Requirements Traced:	FR-NBH-02 to FR-NBH-10
Special Requirements:	NFR-DATA-01 (provenance); NFR-DATA-02 (31-day freshness threshold); NFR-DATA-04 (de-duplicated facility records).
Assumptions:	Distances are measured from the neighbourhood reference point (straight line).
Notes and Issues:	Implemented in client/src/pages/Profile.tsx, GET /neighbourhoods/:id/facilities and server/src/neighbourhoods/dedupe.ts.

UC-5.1 View Map and Nearby Places

Use Case ID:	UC-5.1
Use Case Name:	View Map and Nearby Places
Actor(s):	Visitor (primary); Configured Public Data Sources and OneMap (supporting)
Description:	Shows the neighbourhood and its matched facilities on an interactive map with distinguishable category markers.
Preconditions:	1. A neighbourhood profile is open (UC-4.1).
Postconditions:	• The map shows the neighbourhood boundary, reference point and categorised facility markers.
Priority:	Must
Frequency of Use:	High
Flow of Events:	1. Visitor opens the map on the neighbourhood profile. 2. System draws the OneMap base map with the neighbourhood boundary and reference point. 3. System plots the matched facilities within the radius with a distinct marker for each category. 4. Visitor toggles map layers or changes the radius. 5. System updates the markers and counts. 6. Visitor selects a marker; System shows the facility details and distance.
Alternative Flows:	• AF-3a Facility without coordinates: it is listed but not plotted, with a note. • AF-3b Overlapping markers: each remains individually selectable.
Exceptions:	• Base map unavailable: the facility list and counts remain usable.
Includes / Extends:	None
Requirements Traced:	FR-NBH-02 to FR-NBH-09; FR-TRV-01
Special Requirements:	NFR-ACC-01 (keyboard-accessible map controls); NFR-REL-03 (one dependency failing does not block others).
Assumptions:	None.
Notes and Issues:	Implemented in client/src/pages/Profile.tsx (map layers) and client/src/components/map.tsx. Issue for review: SRS §7.3 links OneMap to UC-5.1 (base map), but the §7.5 catalogue lists only Visitor and data sources. The diagram follows §7.3; align the catalogue.

UC-5.2 Check Commute Route

Use Case ID:	UC-5.2
Use Case Name:	Check Commute Route
Actor(s):	Visitor (primary); OneMap (supporting)
Description:	Requests a route and travel estimate from the neighbourhood to a priority destination for a supported travel mode, optionally asking for a barrier-free route.
Preconditions:	1. A neighbourhood profile is open. 2. A destination is chosen (saved priority destination or entered address).
Postconditions:	• A route with distance, duration, provider and retrieval time is shown, or an explained unavailable state.
Priority:	Must
Frequency of Use:	Medium
Flow of Events:	1. Visitor opens Commute & routes. 2. Visitor chooses a priority destination and a travel mode, and optionally ticks Barrier-free routes only. 3. System checks that the mode is supported by the routing provider. 4. System requests the route from OneMap. 5. System shows the route on the map with distance, duration, provider and retrieval time.
Alternative Flows:	• AF-3a Unsupported mode: System explains the rejection; return to step 2. • AF-4a Barrier-free route requested: System states that step-free coverage is unavailable from the provider and advises checking lift and ramp availability with LTA or the station operator. • AF-4b OneMap finds no route: System shows a "no route" state with a labelled straight-line estimate.
Exceptions:	• OneMap unavailable, timed out or not configured: System shows a labelled straight-line estimate instead; the rest of the profile still works.
Includes / Extends:	None
Requirements Traced:	FR-TRV-02 to FR-TRV-05
Special Requirements:	NFR-REL-03 (routing failure is isolated); NFR-DATA-01 (provider and retrieval time shown).
Assumptions:	None.
Notes and Issues:	Implemented in client/src/pages/Commute.tsx, GET /neighbourhoods/:id/route and server/src/travel/route.ts and onemap.ts.

UC-6.1 Calculate Family Suitability Score

Use Case ID:	UC-6.1
Use Case Name:	Calculate Family Suitability Score
Actor(s):	None (included behaviour)
Description:	Computes an explainable 0 to 100 family suitability score from eight criteria using the user's weights and the versioned scoring configuration.
Preconditions:	1. Invoked by UC-4.1 or UC-6.2 with a neighbourhood and a set of weights (saved, session or default).
Postconditions:	• A total score is returned with each criterion's value, normalised value, weight and contribution, any missing-data notes, and the configuration and dataset versions used.
Priority:	Must
Frequency of Use:	Very high (every profile and comparison)
Flow of Events:	1. The base use case supplies the neighbourhood and the weights. 2. System gathers evidence for the eight criteria from the local validated data. 3. System normalises each value with its criterion's rule (facility count, nearest distance or commute minutes). 4. System excludes criteria that lack data and re-normalises the remaining weights. 5. System computes each contribution and the total, rounding an exact 0.5 upward. 6. System returns the total with the itemised breakdown and version stamps.
Alternative Flows:	• AF-4a All weights are zero: System weights every available criterion equally and discloses this. • AF-4b No criterion has data: the score is shown as unavailable with an explanation.
Exceptions:	• None.
Includes / Extends:	Included by UC-4.1 and UC-6.2.
Requirements Traced:	FR-DEC-01 to FR-DEC-03
Special Requirements:	NFR-DATA-03 (identical inputs and versions reproduce the same score); NFR-TRANS-01 (itemised explanation); NFR-MAINT-01 (rules kept in versioned configuration).
Assumptions:	None.
Notes and Issues:	Implemented in server/src/scoring/score.ts (calculateScore()) and server/src/scoring/config.ts (SCORING_CONFIG).

UC-6.2 Compare Neighbourhoods

Use Case ID:	UC-6.2
Use Case Name:	Compare Neighbourhoods
Actor(s):	Visitor
Description:	Compares two to four selected neighbourhoods side by side using the same weights and radius.
Preconditions:	1. At least two neighbourhoods have been selected for comparison.
Postconditions:	• A side-by-side comparison of scores and family information is displayed.
Priority:	Must
Frequency of Use:	Medium
Flow of Events:	1. Visitor marks two to four neighbourhoods for comparison from results, profiles or the shortlist. 2. Visitor opens Compare neighbourhoods. 3. System validates the selection. 4. System calculates each neighbourhood's score with the same weights and radius (includes UC-6.1). 5. System shows them side by side: suitability match, childcare and kindergartens, schools, daily needs, public transport, accessibility and commute estimate. 6. Visitor changes the radius or weights; System recalculates every column consistently.
Alternative Flows:	• AF-3a Fewer than two or more than four selected: System shows "Choose two to four neighbourhoods to compare."; return to step 1. • AF-3b The same neighbourhood selected twice: System counts it once. • AF-5a A criterion lacks data for one neighbourhood: System marks it in that column.
Exceptions:	• None.
Includes / Extends:	Includes UC-6.1.
Requirements Traced:	FR-DEC-04; FR-DEC-01 to FR-DEC-03
Special Requirements:	NFR-PERF-02 (comparison response time); NFR-TRANS-01 and NFR-TRANS-02.
Assumptions:	None.
Notes and Issues:	Implemented in client/src/pages/Compare.tsx and GET /compare.

UC-6.3 Manage Shortlist and Notes

Use Case ID:	UC-6.3
Use Case Name:	Manage Shortlist and Notes
Actor(s):	Registered User
Description:	Lets a user save up to ten neighbourhoods to a shortlist, remove them, and keep a private note of up to 500 characters on each.
Preconditions:	1. The user is signed in.
Postconditions:	• The shortlist and notes are saved and appear again after the next sign-in.
Priority:	Must
Frequency of Use:	Medium
Flow of Events:	1. User selects Add to shortlist on a neighbourhood. 2. System checks for a duplicate and the ten-entry limit, then saves the entry. 3. User opens Your shortlist. 4. System lists the saved neighbourhoods with their current scores. 5. User writes, edits or clears a private note. 6. System validates and saves the note. 7. User removes an entry; System deletes it and confirms.
Alternative Flows:	• AF-1a Visitor not signed in: System prompts the visitor to sign in or create an account. • AF-2a Neighbourhood already saved: System shows it as already saved and creates no duplicate. • AF-2b An eleventh entry: System rejects it and explains the capacity limit. • AF-6a Note longer than 500 characters: System shows a validation error and keeps the previous note.
Exceptions:	• Session expired: System asks the user to sign in again; nothing is lost.
Includes / Extends:	None
Requirements Traced:	FR-DEC-05 to FR-DEC-07
Special Requirements:	NFR-SEC-05 (notes are validated and rendered as plain text); NFR-PRIV-01 (notes are deleted with the account).
Assumptions:	Notes are private to the owner.
Notes and Issues:	Implemented in client/src/pages/Shortlist.tsx and the /shortlist endpoints. Issue for review: the prototype stores the shortlist per household workspace (SaaS extension); the SRS describes it per user.

UC-7.1 Monitor Automatic Data Synchronisation

Use Case ID:	UC-7.1
Use Case Name:	Monitor Automatic Data Synchronisation
Actor(s):	Data Administrator (primary); Configured Public Data Sources (supporting)
Description:	Lets an administrator review the status and history of the automatic public-data synchronisation. Synchronisation itself runs automatically (FR-ADM-03); there is no manual sync action.
Preconditions:	1. The user is signed in with the Data Administrator role.
Postconditions:	• Source status and synchronisation history are displayed; no data is changed (read-only).
Priority:	Must
Frequency of Use:	Low (routine check)
Flow of Events:	1. Administrator opens the Data sync dashboard. 2. System verifies the administrator role. 3. System shows each source's status, latest successful and latest attempted synchronisation, record count and freshness. 4. Administrator filters the synchronisation history by source or date. 5. System lists the runs newest first with start and end time, outcome, records added, changed and removed, and a sanitised error summary.
Alternative Flows:	• AF-2a Signed-in user without the administrator role: System denies access (403). • AF-2b Not signed in: System asks the user to sign in. • AF-3a A source's latest run failed: System shows the failure while the previous validated dataset stays active.
Exceptions:	• None.
Includes / Extends:	None
Requirements Traced:	FR-ADM-01, FR-ADM-02, FR-ADM-04 (background: FR-ADM-03, FR-RES-01)
Special Requirements:	NFR-SEC-03 (role check at UI and API level); error summaries contain no secrets.
Assumptions:	Synchronisation runs daily at 04:00 Singapore time (SYNC_CRON) and at start-up when a source is due.
Notes and Issues:	Implemented in client/src/pages/Admin.tsx and /admin/sources, /admin/history, /admin/metrics (server/src/synchronisation/routes.ts).